

SOFTWARE ARCHITECTURE DESIGN REPORT

Intelligent AI-Based Warehouse Automation and Inventory Tracking Platform

CO2 ASSESSMENT TOOL 2 ARCHITECTURE DESIGN REPORT

NAME: GAYANA K R

REG NO: 192471039

COURSE NAME: SOFTWARE ENGINEERING

COURSE CODE: CSA1017

1. INTRODUCTION

Warehouses today handle a growing volume of goods, and manual tracking methods simply can't keep pace. Errors in stock counts, wasted floor space, and slow order picking are common problems in warehouses that still rely on manual entry or basic barcode systems. This report looks at how an intelligent warehouse platform built around RFID, IoT sensors, autonomous robots, and AI-based storage decisions could be structured, and why a particular architecture style fits this kind of system best.

2. SYSTEM REQUIREMENTS ANALYSIS

2.1 SYSTEM OBJECTIVES

Based on the problem statement, the platform needs to:

- Track inventory automatically using RFID and IoT devices, without manual counting.
- Decide where items should be stored using AI, so warehouse space is used efficiently.
- Show inventory movement as it happens, not after the fact.
- Speed up how orders are picked, packed, and shipped.
- Catch and reduce mismatches between recorded stock and actual stock.
- Produce reports that help managers see how the warehouse is performing.
- Support RFID tagging as the core method of identifying items.
- Overall, raise productivity while keeping running costs down.

2.2 FUNCTIONAL REQUIREMENTS

These are the things the system must actually do:

- Read RFID tags and IoT sensor data continuously and update inventory records automatically.
- Recommend storage locations for incoming goods based on size, demand frequency, and available space.
- Track the position of AMRs (robots) and coordinate their movement for picking and restocking.
- Update order status in real time as items are picked and moved through the warehouse.
- Flag discrepancies between physical counts and system records automatically.
- Generate dashboards and periodic reports on stock levels, throughput, and utilization.
- Allow warehouse staff to search, view, and manually adjust inventory when needed.
- Send alerts for low stock, misplaced items, or equipment issues.

2.3 NON-FUNCTIONAL REQUIREMENTS

These describe how well the system needs to perform, rather than what it does:

- The system should process sensor and RFID events with low delay, ideally within a second or two, so the dashboard reflects reality closely.

- It should scale to handle thousands of RFID reads per minute during peak periods, like large shipments or seasonal sales.
- Inventory data must stay accurate and consistent even when multiple devices update it at the same time.
- The platform should stay available around the clock, since warehouse operations often run in multiple shifts.
- Access should be controlled — floor staff, managers, and external partners should only see and do what their role permits.
- The system should be easy to extend later, for example adding a new type of sensor or a new AI model, without rewriting everything.
- Data should be recoverable after a failure — if a service crashes mid-update, no inventory event should be permanently lost.
- The user interface (dashboard and mobile app) should stay usable even during short network interruptions on the warehouse floor, syncing once connectivity returns.

2.4 KEY QUALITY ATTRIBUTES

A few quality attributes matter more than others for this kind of system, given how much real-time data it handles and how central it is to daily operations:

- **Scalability** – the number of connected devices and data volume will grow as the warehouse expands or adds more RFID points.
- **Reliability** – inventory data has to be correct; a wrong count can mean a failed order or a lost sale.
- **Availability** – warehouse floors run continuously, so downtime directly stops physical operations.
- **Performance** – location tracking and order updates need to happen close to instantly to be useful.
- **Security** – the system connects to ERP and logistics partners, so data exchange needs to be protected.
- **Maintainability** – warehouse tech changes often (new robots, new sensor types), so the system should be easy to update piece by piece.

2.5 ASSUMPTIONS AND CONSTRAINTS

A few practical assumptions shaped the design choices below:

- The warehouse already has (or will install) RFID readers and a basic wireless/wired network covering the floor — the platform doesn't need to design the physical network itself, just integrate with it.
- AMRs are assumed to expose some kind of API or messaging interface for receiving tasks and reporting position; the platform coordinates them rather than controlling their low-level motion.
- The organization likely already runs an ERP or WMS system, so the platform is designed to integrate with an existing system rather than replace it outright.
- Internet connectivity may occasionally drop on the warehouse floor, so edge components should tolerate brief disconnections without losing data.

3. SELECTED SOFTWARE ARCHITECTURE

A hybrid approach works best here: a layered structure at a high level (edge devices, gateway, application, data, presentation), combined with a microservices style within the application layer, tied together with an event-driven messaging layer.

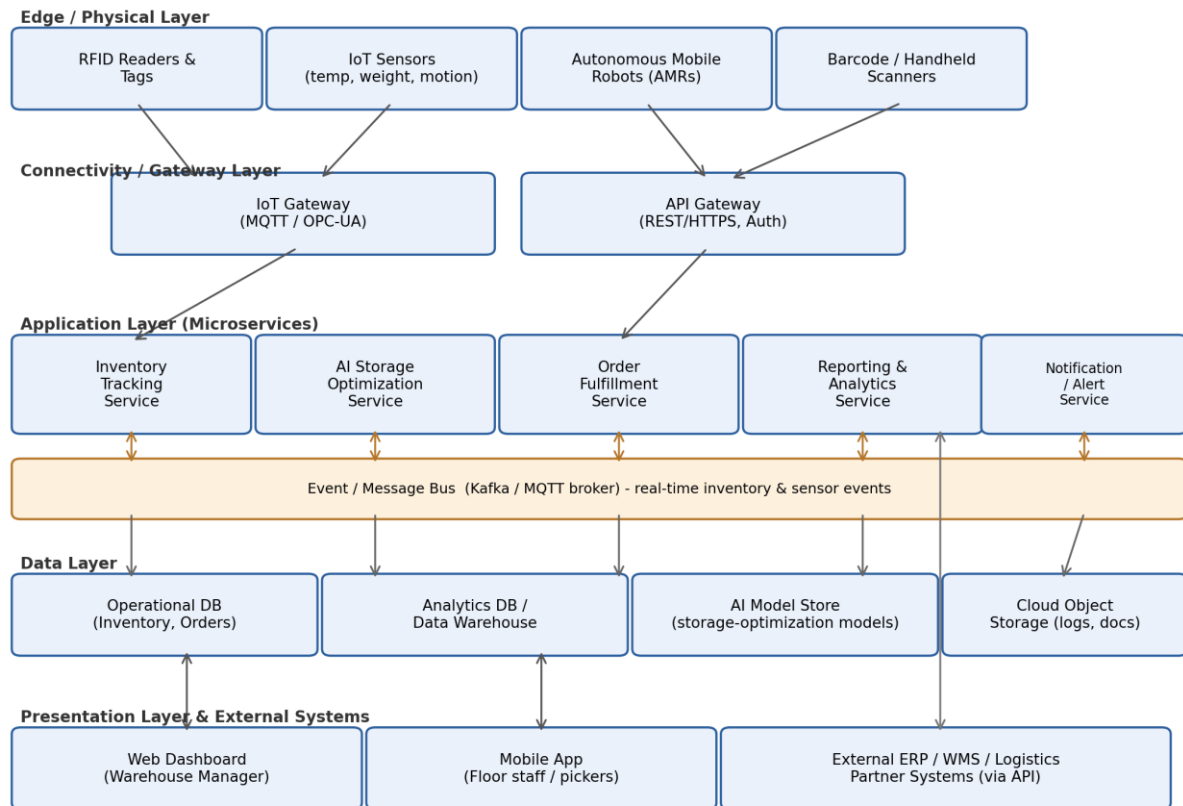
The reason for this combination is fairly practical. RFID and IoT data arrives constantly and unpredictably — this is naturally an event stream, not a request-response pattern, so an event-driven backbone (using something like Kafka or an MQTT broker) fits well. At the same time, the platform has several distinct responsibilities — tracking inventory, running AI storage optimization, managing orders, generating reports — and these don't need to be one giant application. Splitting them into separate microservices means each one can be scaled, updated, or replaced on its own. For example, if the AI storage optimization model needs retraining or a performance upgrade, that shouldn't require touching the order fulfillment service at all.

A pure monolithic or simple client-server design would struggle here — a single application handling both high-frequency sensor ingestion and heavier AI computation tends to become a bottleneck, and it's harder to scale specific parts independently. A pure microservices architecture without the event bus, on the other hand, would need every service to poll or call each other directly, which doesn't handle bursts of RFID/IoT data well. Combining the two — layered organization, microservices for logic, and an event bus for real-time data flow — gives a reasonable balance of structure and flexibility.

4. SOFTWARE ARCHITECTURE DIAGRAM

The diagram below shows the overall structure, from the physical devices on the warehouse floor up to the dashboards and external systems.

Intelligent Warehouse Automation Platform - Architecture Overview



5. COMPONENTS AND THEIR INTERACTIONS

5.1 EDGE / PHYSICAL LAYER

This is where data originates: RFID readers and tags identify items as they move, IoT sensors capture conditions like temperature or weight, AMRs move goods around the floor, and handheld scanners let staff confirm actions manually. These devices send raw readings upward rather than making decisions themselves.

5.2 CONNECTIVITY / GATEWAY LAYER

The IoT gateway collects sensor and RFID readings using lightweight protocols such as MQTT, cleans up the data a bit (filtering noise, formatting), and forwards it into the message bus. The API gateway handles structured requests — from robots reporting status, or from external systems calling in — and takes care of authentication before passing requests to the right microservice.

5.3 APPLICATION LAYER (MICROSERVICES)

Each microservice owns one part of the problem:

- **Inventory Tracking Service** – keeps the live record of what's in stock and where, updated as RFID/IoT events come in.

- **AI Storage Optimization Service** – uses historical movement and demand data to suggest or assign storage locations for new stock.
- **Order Fulfillment Service** – manages the picking and packing workflow, coordinating with AMRs and updating order status.
- **Reporting & Analytics Service** – aggregates data for dashboards and scheduled reports on productivity and utilization.
- **Notification / Alert Service** – watches for conditions like low stock or discrepancies and pushes alerts to staff.

These services don't call each other directly most of the time. Instead, they publish and subscribe to events on the message bus — for instance, when the Inventory Service records a new item location, it publishes an event that the Reporting Service and Notification Service can both pick up independently.

5.4 EVENT / MESSAGE BUS

This sits between the application services and acts as the nervous system of the platform. It decouples producers of data (sensors, robots) from consumers (services, dashboards), which means a spike in RFID reads doesn't overwhelm any single service — the bus buffers and distributes the load.

5.5 DATA LAYER

Different types of data go to different stores: the operational database holds current inventory and order records that need to be read and written quickly; the analytics database/data warehouse holds historical data for trend analysis and reporting; the AI model store holds the trained models used for storage optimization; and cloud object storage keeps logs, documents, and larger files.

5.6 PRESENTATION LAYER AND EXTERNAL SYSTEMS

Warehouse managers use a web dashboard to see stock levels, utilization, and reports. Floor staff and pickers use a mobile app for day-to-day tasks like confirming picks or scanning items. The platform also connects to external ERP, WMS, or logistics partner systems through the API gateway, so inventory and order data can flow into the broader supply chain.

5.7 OVERALL WORKFLOW

A simple example: an item enters the warehouse and is tagged with RFID. The RFID reader picks up the tag and sends the reading through the IoT gateway to the message bus. The Inventory Tracking Service consumes this event and updates the record. The AI Storage Optimization Service, watching for new-item events, suggests a storage location and passes this to an AMR through the Order Fulfillment Service, which coordinates the move. Once stored, the update is reflected on the manager's dashboard and logged for reporting, all within a few seconds of the item being scanned.

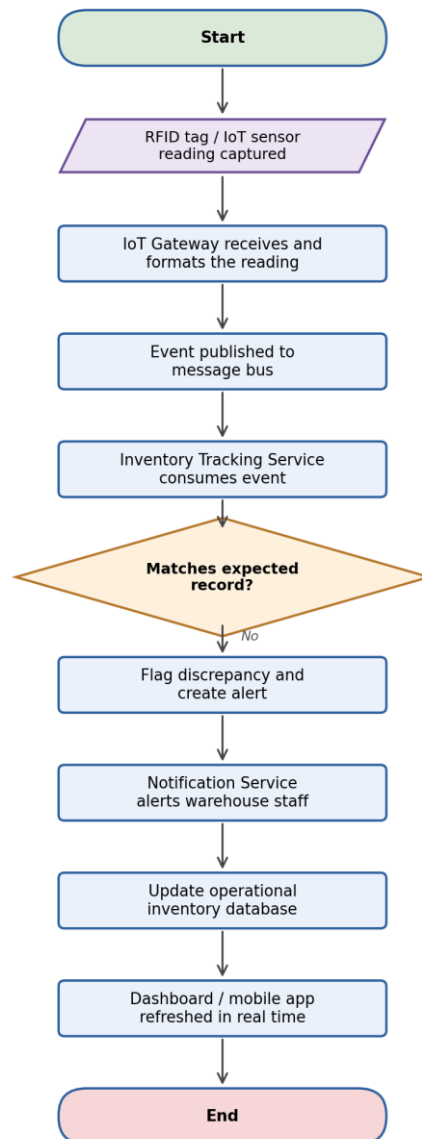
6. PROCESS FLOWCHARTS

The component diagram in Section 4 shows the static structure of the system — what the pieces are and how they connect. The flowcharts below complement that by showing the dynamic side: the step-by-step sequence of events for three of the platform's core processes.

6.1 RFID/IOT INVENTORY UPDATE PROCESS

This is the most frequent process in the system, since it runs every time a tagged item is scanned or a sensor sends a reading. It's also where discrepancy detection happens, so it directly supports the objective of reducing inventory errors.

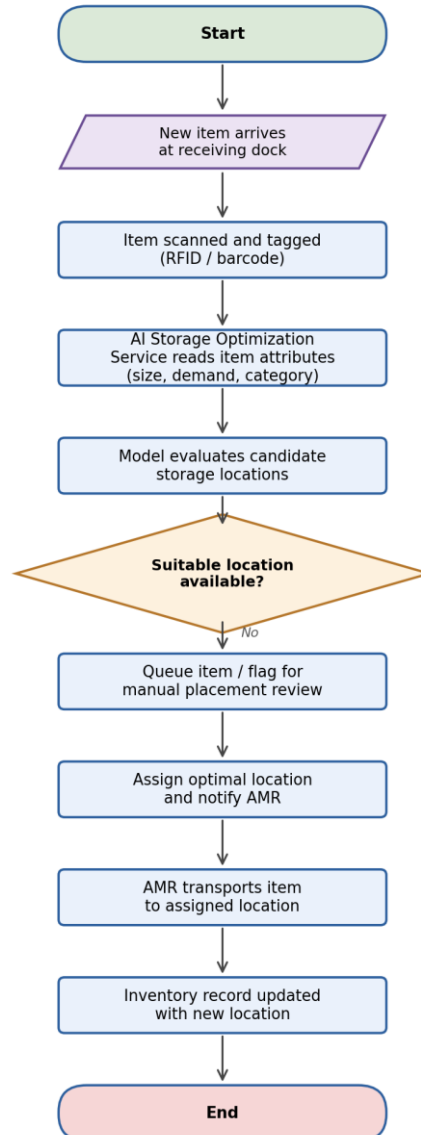
Flowchart 1: RFID/IoT Inventory Update Process



6.2 AI-BASED STORAGE ALLOCATION PROCESS

This process runs whenever new stock arrives and needs a storage location. The AI service looks at attributes like item size, category, and how often it's likely to be picked (demand frequency) to recommend a location that keeps fast-moving items easy to reach and makes better use of available space.

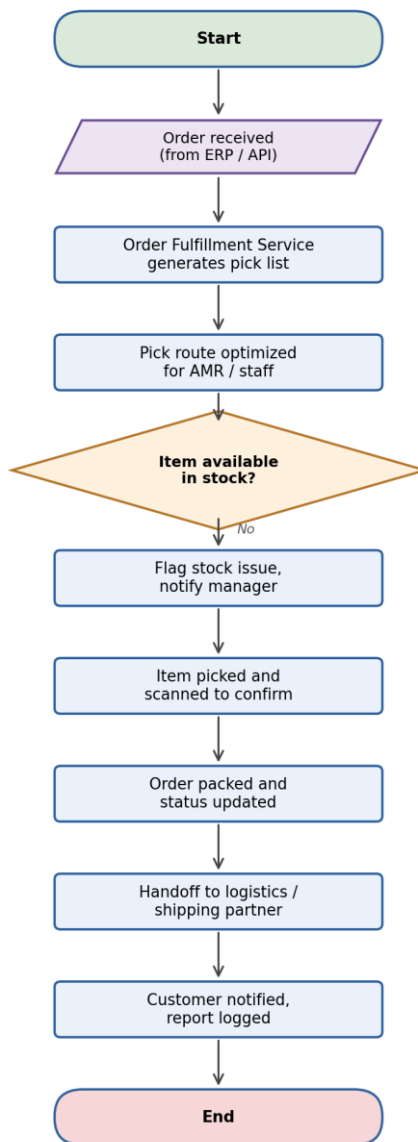
Flowchart 2: AI-Based Storage Allocation Process



6.3 ORDER FULFILLMENT PROCESS

This process starts once an order comes in from an external system (like an ERP or e-commerce platform) and ends when the order is handed off to a shipping or logistics partner. It's the process most directly tied to customer satisfaction, since delays or stock issues here affect delivery times.

Flowchart 3: Order Fulfillment Process



7. DISCUSSION OF QUALITY ATTRIBUTES

Attribute	How the architecture addresses it
Scalability	Microservices can be scaled independently — for example, adding more instances of the Inventory Service during peak intake, without touching the Reporting Service. The message bus also absorbs bursts of RFID/IoT traffic instead of overwhelming any single component.
Security	The API gateway centralizes authentication and access control for external and internal requests, so individual services don't each have to reimplement it. Role-based access limits what floor staff, managers, and partner systems can see or do.

Attribute	How the architecture addresses it
Performance	Event-driven processing means services react to data as it arrives rather than waiting on scheduled polling. The operational database is kept separate from the analytics database, so heavy reporting queries don't slow down real-time inventory updates.
Reliability	Because services communicate through the message bus, if one service is temporarily down, events queue up rather than getting lost, and processing catches up once the service is back.
Maintainability	Each microservice is a separate, smaller codebase with a clear responsibility, so a team can update the AI optimization logic, for instance, without needing to understand or risk breaking the order fulfillment code.
Availability	Services are decoupled and can be deployed with redundancy (multiple running instances) so that the failure of one instance doesn't take down the whole platform. The layered design also means a gateway or edge issue doesn't necessarily bring down the data layer.

8. JUSTIFICATION OF ARCHITECTURAL DECISIONS

The main decision was to avoid a single monolithic backend and instead split responsibilities into services connected through an event bus. This does add some complexity — there are more moving parts to deploy and monitor compared to one application — but for a warehouse system with constant sensor input and several fairly independent functions (tracking, AI recommendations, order handling, reporting), it fits the workload better than one tightly coupled application would.

One trade-off worth noting is operational overhead. Running and monitoring several services and a message broker takes more setup than a simple client-server app, and the team needs some infrastructure experience to manage it well. In exchange, the platform gains the ability to scale specific bottlenecks (usually the inventory and AI services under heavy load) without over-provisioning the entire system, and it becomes easier to swap out or upgrade individual pieces — such as replacing the AI storage model or adding a new type of sensor — over time.

For future growth, this structure also supports adding new capabilities without much disruption. A new microservice (say, for predictive maintenance on AMRs) could simply subscribe to existing events on the message bus rather than requiring changes to the services already running. Integration with additional external systems, like a new logistics partner, is handled through the API gateway, keeping that concern isolated from the core inventory logic. This should make the platform reasonably future-proof as the warehouse's needs change.

9. SUGGESTED TECHNOLOGY STACK

This isn't a strict requirement, but it helps to ground the architecture in realistic tools. The table below lists reasonable technology choices for each layer, based on what's commonly used for this kind of system.

Layer	Example technologies
Edge / Physical	Passive/active RFID tags and readers, industrial IoT sensors, AMR platforms (e.g., ROS-based robots)
Connectivity / Gateway	MQTT broker or OPC-UA for device data, NGINX or Kong as API gateway, OAuth2/JWT for authentication
Application (Microservices)	Node.js, Java (Spring Boot), or Python (FastAPI) services, containerized with Docker, orchestrated with Kubernetes
Event / Message Bus	Apache Kafka or RabbitMQ for event streaming between services
AI / Storage Optimization	Python with scikit-learn or TensorFlow for the optimization model, retrained periodically on historical movement data
Data Layer	PostgreSQL or MySQL for operational data, a data warehouse such as Snowflake or BigQuery for analytics, Amazon S3 or similar for object storage
Presentation	React or Angular for the web dashboard, Flutter or React Native for the mobile app
Infrastructure	Cloud hosting (AWS, Azure, or GCP), with monitoring via Prometheus/Grafana and centralized logging (ELK stack)

None of these choices are fixed — the architecture doesn't depend on a specific vendor or tool, which is one of the benefits of keeping the layers loosely coupled through APIs and the message bus.

10. CONCLUSION

The proposed architecture — a layered structure with microservices and an event-driven backbone — is built around the reality that warehouse data is constant, high-volume, and needs quick responses. It separates concerns cleanly enough that the system can grow, whether that means more devices, more warehouses, or new AI capabilities, while keeping day-to-day inventory tracking accurate and responsive.